

✓ MIS780 Advanced AI For Business - Assessment 2 - T2 2025

Task 2 – Image-Based Recyclable Material Classification

Student Name: Leon Wilson **Student ID:** 224142965

Table of Content

1. [Executive Summary](#)
2. [Data Preprocessing](#)
3. [Predictive Modeling](#)
4. [Experiments Report](#)
5. [Role of GenAI](#)

✓ 1. Executive Summary

The recycling industry faces challenges in efficiently sorting and classifying different types of materials, which is critical for sustainable waste management. This task focuses on developing an AI-based solution for automatic classification of recyclable materials using a dataset of 2,691 real images of six material types: cardboard, clothes, paper, metal, green glass, and white glass.

The study involves exploratory data analysis, CNN model development, and performance evaluation. Multiple CNN architectures were tested, including baseline MLP, standard CNNs, deeper CNNs with additional layers, and regularized CNNs. Models were compared using key metrics such as accuracy, Kappa coefficient, precision, recall, and F1-score to identify the most effective architecture.

The results demonstrated that deeper CNN architectures with multiple convolutional layers, sufficient kernel sizes, and proper regularization achieved the highest performance. Among the six material classes, white glass and metal were observed to have relatively lower classification performance, likely due to variations in transparency, color similarity, or image backgrounds.

This AI solution can be applied in real-world recycling operations to automate the sorting of recyclable materials, reduce manual labor, improve efficiency, and promote sustainable practices. Careful attention to data quality, model architecture, and deployment considerations is essential for achieving reliable performance in real-world applications.

✓ 2. Data Preprocessing

```
%%html
<style>table {float:left}</style>
<style>img {float:left}</style>
```



```
import os
import random
import pandas as pd
import matplotlib.pyplot as plt
import math
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import classification_report, cohen_kappa_score, confusion_matrix, ConfusionMatrixDisplay

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)

set_seed(42)
```

```
def plot_images(ims, figsize=(12,12), cols=1, interp=False, titles=None):
    if type(ims[0]) is np.ndarray:
        if (ims.shape[-1] != 3):
            ims = ims[:, :, :, 0]
```

```
f = plt.figure(figsize=figsize)
rows = len(ims)//cols if len(ims)%cols == 0 else len(ims)//cols + 1
for i in range(len(ims)):
    sp = f.add_subplot(rows, cols, i+1)
    sp.axis('Off')
    if titles is not None:
        sp.set_title(titles[i], fontsize=12)
    plt.imshow(ims[i], interpolation=None if interp else 'none')
plt.tight_layout()
plt.show()
```

```
def plot_hist(history, xsize=6, ysize=5):
    fig_size = plt.rcParams["figure.figsize"]
    plt.rcParams["figure.figsize"] = [xsize, ysize]
    h = history.history
    ks = list(h.keys())
    n2 = math.floor(len(ks)/2)
    train_keys = ks[0:n2]
    valid_keys = ks[n2:2*n2]
    for i in range(n2):
        plt.plot(h[train_keys[i]])
        plt.plot(h[valid_keys[i]])
        plt.title('Training vs Validation '+train_keys[i])
        plt.ylabel(train_keys[i])
        plt.xlabel('Epoch')
        plt.legend(['Train', 'Validation'], loc='upper left')
        plt.show()
    plt.rcParams["figure.figsize"] = fig_size
    return
```

```
# Mount Google Drive
drive.mount('/content/drive')
dataset_path = '/content/drive/MyDrive/Colab Notebooks/MIS780/Task 2 Dataset Recyclable Material'
```

```
# Discover class folders
print("Class folders under dataset:")
class_names = sorted(os.listdir(dataset_path))
print(class_names)
num_classes = len(class_names)
```

```
→ Class folders under dataset:
['cardboard', 'clothes', 'green-glass', 'metal', 'paper', 'white-glass']
```

```
# Print the total number of files in each class folder
for cname in class_names:
    folder = os.path.join(dataset_path, cname)
    file_list = [f for f in os.listdir(folder) if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    print(f"Total number of files under {cname} folder are: {len(file_list)}")
```

```
→ Total number of files under cardboard folder are: 417
Total number of files under clothes folder are: 511
Total number of files under green-glass folder are: 449
Total number of files under metal folder are: 430
Total number of files under paper folder are: 479
Total number of files under white-glass folder are: 405
```

```
img_rows, img_cols = 100, 100
channels = 3
```

```
# Load images and labels
data = []
for idx, cname in enumerate(class_names):
    folder = os.path.join(dataset_path, cname)
    for file in os.listdir(folder):
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            img = tf.io.read_file(os.path.join(folder, file))
            img = tf.image.decode_jpeg(img, channels=3)
            img = tf.image.resize(img, (img_rows, img_cols))
            data.append((img, idx))
```

```
# Shuffle and split data (80/20)
random.seed(42)
random.shuffle(data)
split_idx = int(len(data) * 0.8)
```

```
train_data, test_data = data[:split_idx], data[split_idx:]
```

```
# Extract images and labels, convert to numpy arrays
X_train, Y_train = zip(*train_data)
X_test, Y_test = zip(*test_data)
X_train = np.array([img.numpy() for img in X_train])
X_test = np.array([img.numpy() for img in X_test])
Y_train = np.array(Y_train)
Y_test = np.array(Y_test)
```

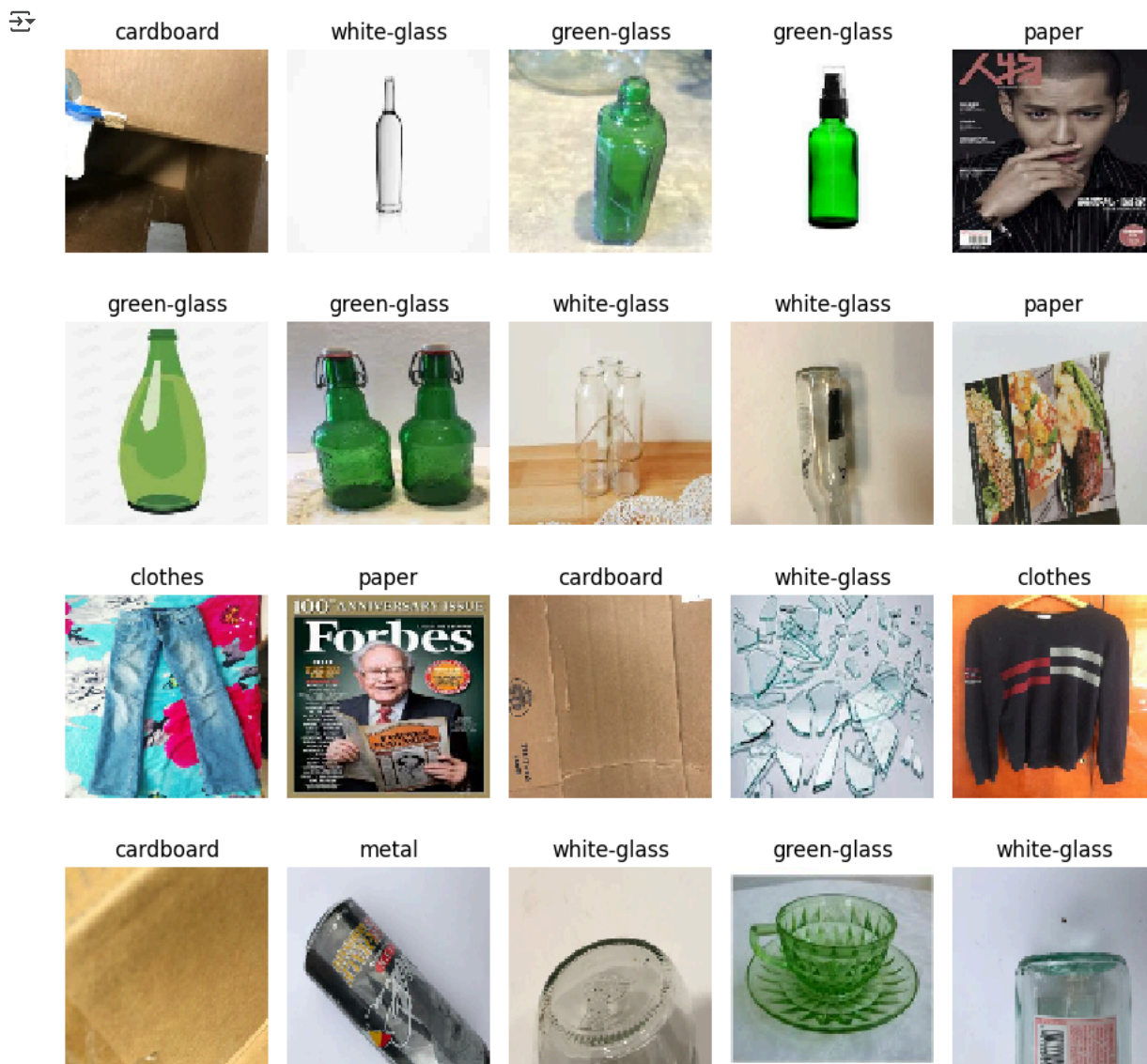
```
# Normalize images
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
```

```
print("Training matrix shape:", X_train.shape)
print("Testing matrix shape:", X_test.shape)
```

↗ Training matrix shape: (2152, 100, 100, 3)
Testing matrix shape: (539, 100, 100, 3)

```
# One-hot encode labels for categorical_crossentropy
Y_train_cat = to_categorical(Y_train, num_classes)
Y_test_cat = to_categorical(Y_test, num_classes)
```

```
# Plot sample images
plot_images(X_train[0:20], cols=5, figsize=[9,9], titles=[class_names[y] for y in Y_train[0:20]])
```



✓ 3. Predictive Modeling

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Conv2D, Flatten
```

```
from tensorflow.keras.layers import MaxPooling2D, Activation, BatchNormalization
from tensorflow.keras.callbacks import TensorBoard, Callback, EarlyStopping
from tensorflow.keras.optimizers import SGD, RMSprop, Adam, Nadam
from tensorflow.keras.losses import categorical_crossentropy
from tensorflow.keras import regularizers
from tensorflow.keras.regularizers import l2
```

```
def model_1(img_rows, img_cols, channels, num_classes):
    model = Sequential()
    model.add(Flatten(input_shape=(img_rows, img_cols, channels)))
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(num_classes, activation='softmax'))
    model.summary()
    return model
```

```
def model_2(img_rows, img_cols, channels, num_classes):
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(img_rows, img_cols, channels)),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    return model
```

```
def model_3(img_rows, img_cols, channels, num_classes):
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(img_rows, img_cols, channels)),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        Conv2D(128, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    return model
```

```
def model_4(img_rows, img_cols, channels, num_classes):
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(img_rows, img_cols, channels),
            kernel_regularizer=l2(0.001)),
        MaxPooling2D((2, 2)),
        Dropout(0.2),
        Conv2D(64, (3, 3), activation='relu', kernel_regularizer=l2(0.001)),
        MaxPooling2D((2, 2)),
        Dropout(0.3),
        Conv2D(128, (3, 3), activation='relu', kernel_regularizer=l2(0.001)),
        MaxPooling2D((2, 2)),
        Dropout(0.4),
        Flatten(),
        Dense(128, activation='relu', kernel_regularizer=l2(0.001)),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    return model
```

```
model = model_4(img_rows, img_cols, channels, num_classes)
model.summary()
```

 /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 98, 98, 32)	896
max_pooling2d_8 (MaxPooling2D)	(None, 49, 49, 32)	0
dropout_8 (Dropout)	(None, 49, 49, 32)	0
conv2d_9 (Conv2D)	(None, 47, 47, 64)	18,496
max_pooling2d_9 (MaxPooling2D)	(None, 23, 23, 64)	0
dropout_9 (Dropout)	(None, 23, 23, 64)	0
conv2d_10 (Conv2D)	(None, 21, 21, 128)	73,856
max_pooling2d_10 (MaxPooling2D)	(None, 10, 10, 128)	0
dropout_10 (Dropout)	(None, 10, 10, 128)	0
flatten_5 (Flatten)	(None, 12800)	0
dense_10 (Dense)	(None, 128)	1,638,528
dropout_11 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 6)	774

Total params: 1,732,550 (6.61 MB)
Trainable params: 1,732,550 (6.61 MB)

```
# Keras callbacks
keras_callbacks = [EarlyStopping(monitor='val_loss', patience=20, verbose=1, restore_best_weights=True)]
```

```
# Compile model
model.compile(
    loss='categorical_crossentropy',
    optimizer=RMSprop(learning_rate=0.001, weight_decay=1e-6),
    metrics=['accuracy']
)

# Train model (using test set as validation)
hist = model.fit(
    X_train, Y_train_cat,
    batch_size=128,
    epochs=100,
    verbose=2,
    validation_data=(X_test, Y_test_cat),
    callbacks=keras_callbacks
)
```



```

epoch 49/100
17/17 - 1s - 52ms/step - accuracy: 0.9108 - loss: 0.4889 - val_accuracy: 0.7607 - val_loss: 0.9307
Epoch 50/100
17/17 - 1s - 70ms/step - accuracy: 0.8880 - loss: 0.5931 - val_accuracy: 0.8386 - val_loss: 0.7031
Epoch 51/100
17/17 - 2s - 90ms/step - accuracy: 0.9112 - loss: 0.5029 - val_accuracy: 0.8442 - val_loss: 0.7086
Epoch 52/100
17/17 - 1s - 64ms/step - accuracy: 0.9126 - loss: 0.4977 - val_accuracy: 0.8163 - val_loss: 0.7535
Epoch 53/100
17/17 - 1s - 61ms/step - accuracy: 0.8801 - loss: 0.5925 - val_accuracy: 0.8367 - val_loss: 0.7206
Epoch 54/100
17/17 - 1s - 77ms/step - accuracy: 0.9354 - loss: 0.4406 - val_accuracy: 0.8256 - val_loss: 0.7484
Epoch 55/100
17/17 - 1s - 48ms/step - accuracy: 0.9205 - loss: 0.4728 - val_accuracy: 0.7996 - val_loss: 0.8490
Epoch 56/100
17/17 - 1s - 57ms/step - accuracy: 0.9071 - loss: 0.5096 - val_accuracy: 0.8071 - val_loss: 0.8636
Epoch 57/100
17/17 - 1s - 49ms/step - accuracy: 0.9201 - loss: 0.4576 - val_accuracy: 0.7904 - val_loss: 0.9554
Epoch 58/100
17/17 - 1s - 48ms/step - accuracy: 0.9187 - loss: 0.4789 - val_accuracy: 0.8516 - val_loss: 0.6800
Epoch 59/100
17/17 - 1s - 48ms/step - accuracy: 0.9126 - loss: 0.5012 - val_accuracy: 0.8497 - val_loss: 0.7039
Epoch 60/100
17/17 - 1s - 74ms/step - accuracy: 0.9201 - loss: 0.4640 - val_accuracy: 0.8033 - val_loss: 0.8455
Epoch 60: early stopping
Restoring model weights from the end of the best epoch: 40.

```

```

# Evaluate on training data
train_score = model.evaluate(X_train, Y_train_cat, verbose=0)
print('Train loss:', round(train_score[0], 4))
print('Train accuracy:', round(train_score[1], 4), '\n')

```

```

# Evaluate on test data
test_score = model.evaluate(X_test, Y_test_cat, verbose=0)
print('Test loss:', round(test_score[0], 4))
print('Test accuracy:', round(test_score[1], 4))

```

```

➡ Train loss: 0.4056
  Train accuracy: 0.9512

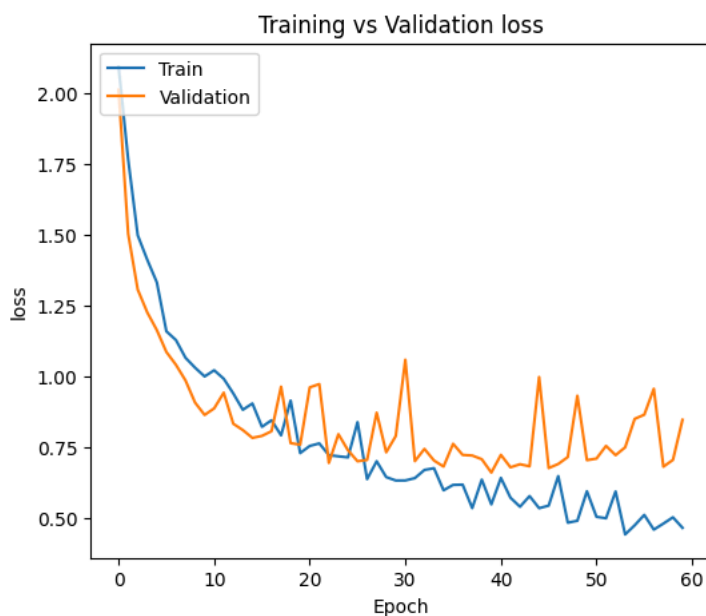
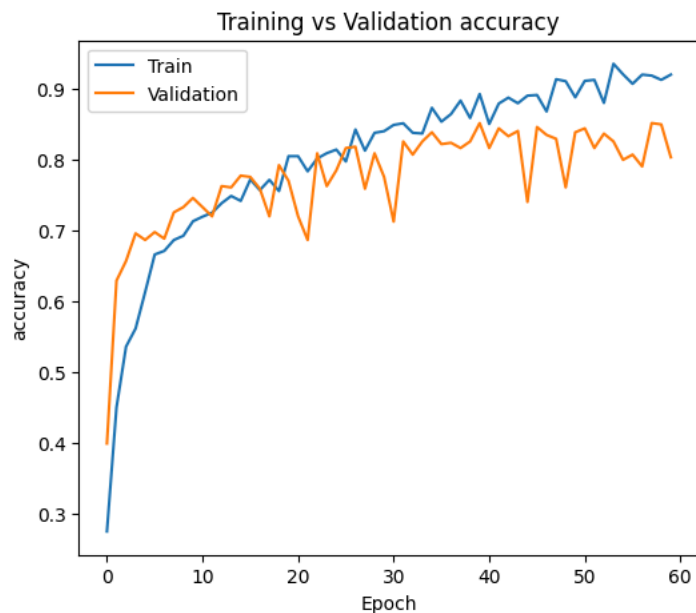
  Test loss: 0.6591
  Test accuracy: 0.8516

```

```

# Plot training history
plot_hist(hist)

```



```
# Make predictions on the test set
y_pred = model.predict(X_test)
y_pred_multiclass = np.argmax(y_pred, axis=1)
y_test_multiclass = np.argmax(Y_test_cat, axis=1)

# Calculate the kappa score
kappa = cohen_kappa_score(y_test_multiclass, y_pred_multiclass)
print("The result of Kappa is :", round(kappa, 3))

# Generate the classification report
report = classification_report(y_test_multiclass, y_pred_multiclass, target_names=class_names)
print("The result of the classification report is: \n", report)
```



17/17 ————— 1s 27ms/step
The result of Kappa is : 0.822
The result of the classification report is:

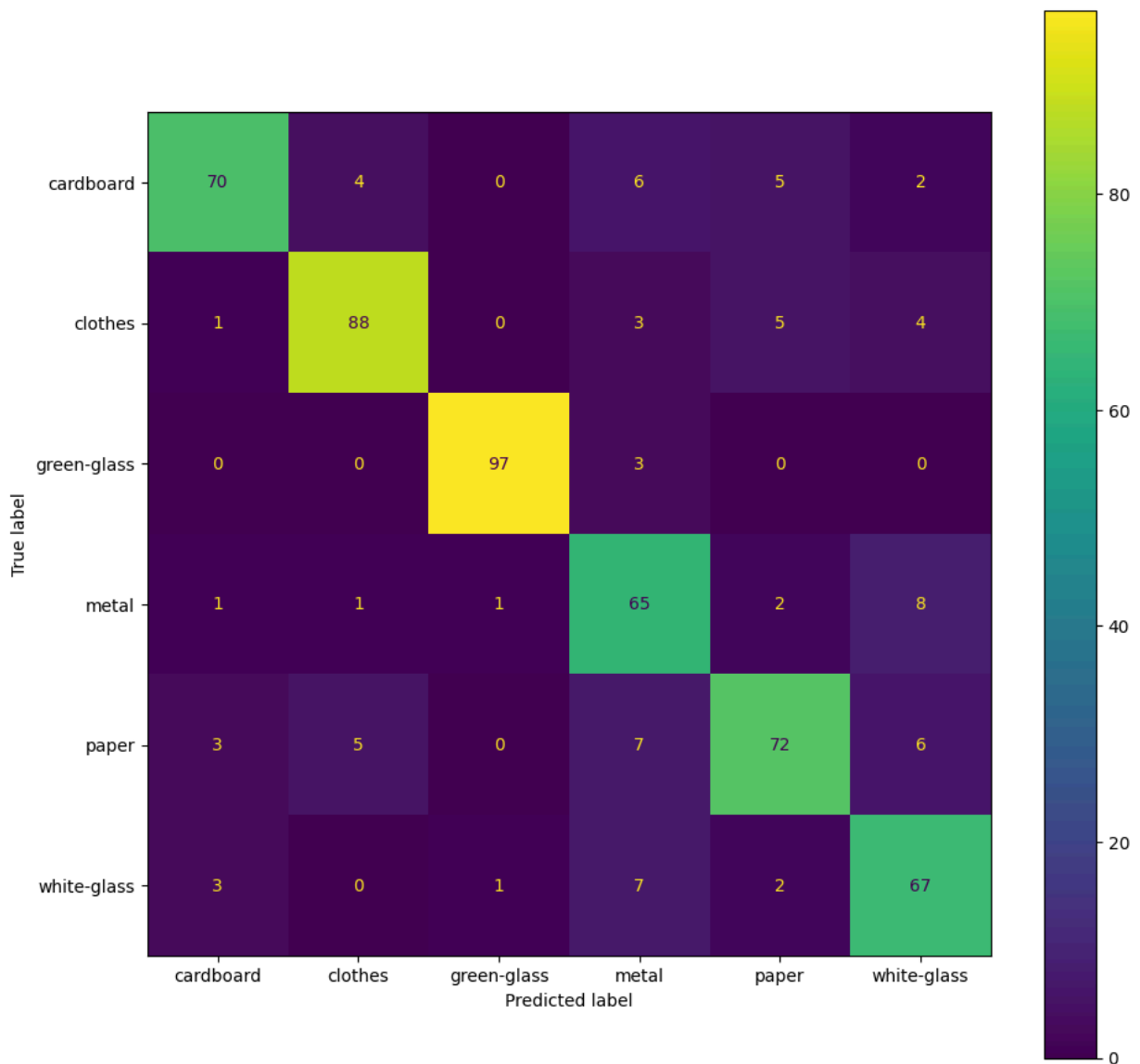
	precision	recall	f1-score	support
cardboard	0.90	0.80	0.85	87
clothes	0.90	0.87	0.88	101
green-glass	0.98	0.97	0.97	100
metal	0.71	0.83	0.77	78
paper	0.84	0.77	0.80	93
white-glass	0.77	0.84	0.80	80
accuracy			0.85	539
macro avg	0.85	0.85	0.85	539
weighted avg	0.86	0.85	0.85	539

```
# Plot confusion matrix
```

```

cm = confusion_matrix(y_test_multiclass, y_pred_multiclass)
display = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
fig = plt.figure(figsize=(11, 11))
ax = fig.subplots()
display.plot(ax=ax)
plt.show()

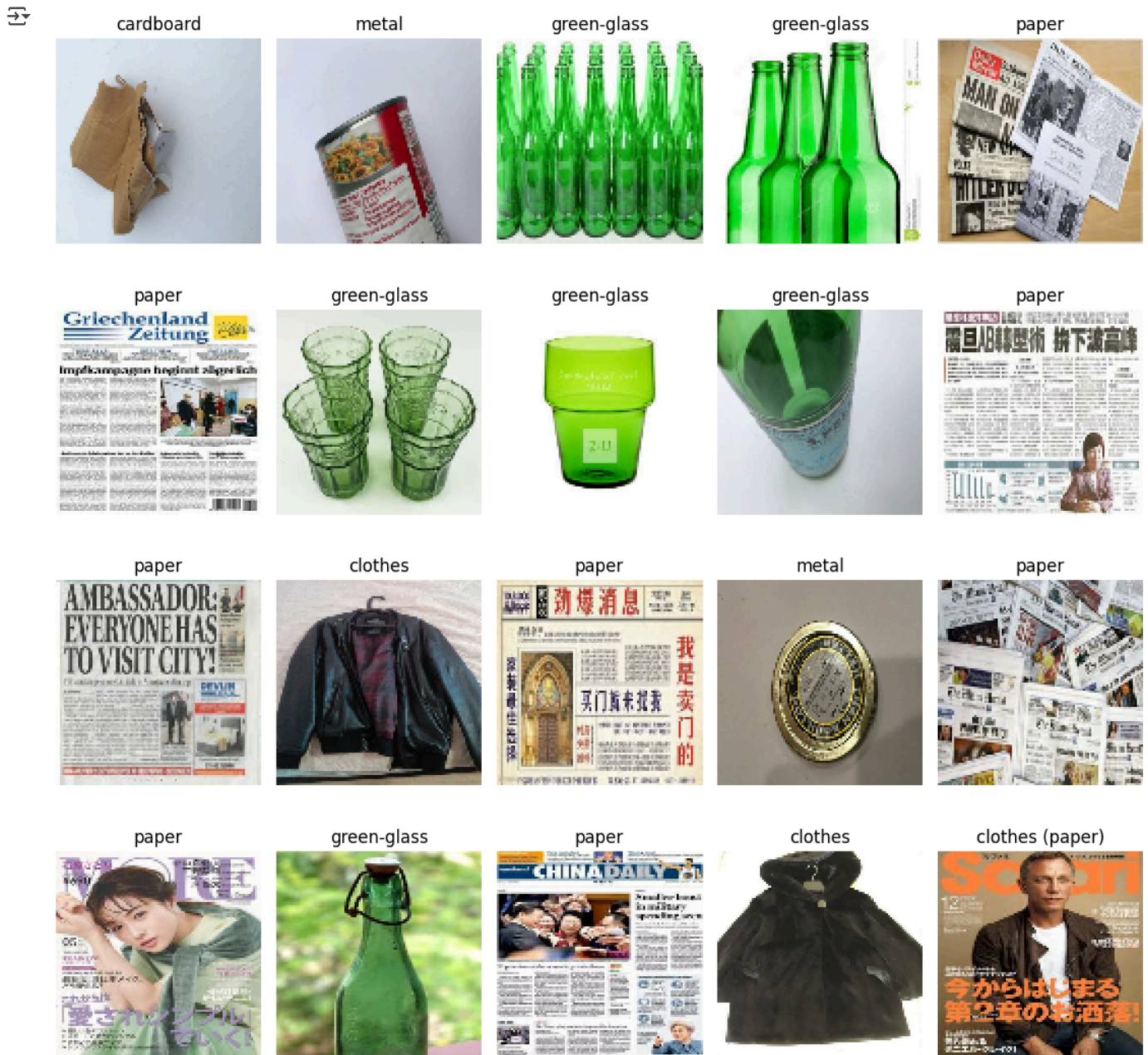
```



```

# Visualize predictions on test images
img_range = range(20)
imgs = X_test[img_range]
true_labels = [class_names[x] for x in y_test_multiclass[img_range]]
pred_labels = [class_names[x] for x in y_pred_multiclass[img_range]]
titles = [pred_labels[i]+' if true_labels[i] == pred_labels[i] else ' ('+true_labels[i]+'') for i in range(len(img_range))]
plot_images(imgs, cols=5, figsize=(11,11), titles=titles)

```

4. Experiments Report

Iteration	Model Architecture	Model Settings	Accuracy	Kappa	Test loss	Train loss
1	MLP (Baseline, no conv) Flatten Dense(128, relu) Dropout(0.2) Dense(num_classes, softmax)	Optimizer: RMSprop Learning Rate: 0.001 Batch Size: 128 Epochs: 100	0.1744	0.002	1.7873	1.7876
2	CNN: Conv2D(32,3x3,relu)->MaxPool2D Conv2D(64,3x3,relu)->MaxPool2D Flatten Dense(128,relu) Dropout(0.5) Dense(num_classes,softmax)	Optimizer: RMSprop Learning Rate: 0.001 Batch Size: 128 Epochs: 100	0.7829	0.739	0.5906	0.1028
3	CNN: Conv2D(32,3x3,relu)->MaxPool2D Conv2D(64,3x3,relu)->MaxPool2D Conv2D(128,3x3,relu)->MaxPool2D Flatten Dense(128,relu) Dropout(0.5) Dense(num_classes,softmax)	Optimizer: RMSprop Learning Rate: 0.001 Batch Size: 128 Epochs: 100	0.8479	0.817	0.4573	0.151
4	CNN + regularization: Conv2D(32,3x3,relu,l2)->MaxPool2D->Dropout(0.2) Conv2D(64,3x3,relu,l2)->MaxPool2D->Dropout(0.3) Conv2D(128,3x3,relu,l2)->MaxPool2D->Dropout(0.4) Flatten Dense(128,relu,l2)->Dropout(0.5) Dense(num_classes,softmax)	Optimizer: RMSprop Learning Rate: 0.001 Batch Size: 128 Epochs: 100	0.8516	0.822	0.6591	0.4056

Iteration	Class	Precision	Recall	F1-score	Support
1	cardboard	0.00	0.00	0.00	87
	clothes	1.00	0.01	0.02	101
	green-glass	0.00	0.00	0.00	100
	metal	0.00	0.00	0.00	78
	paper	0.18	1.00	0.30	93
	white-glass	0.00	0.00	0.00	80
	accuracy			0.17	539
	macro avg	0.20	0.17	0.05	539
	weighted avg	0.22	0.17	0.06	539
2	cardboard	0.83	0.75	0.79	87
	clothes	0.77	0.87	0.82	101
	green-glass	0.96	0.97	0.97	100
	metal	0.61	0.74	0.67	78
	paper	0.79	0.68	0.73	93
	white-glass	0.72	0.64	0.68	80
	accuracy			0.78	539
	macro avg	0.78	0.77	0.77	539
	weighted avg	0.79	0.78	0.78	539
3	cardboard	0.90	0.87	0.89	87
	clothes	0.94	0.79	0.86	101
	green-glass	0.97	0.97	0.97	100
	metal	0.77	0.69	0.73	78
	paper	0.85	0.88	0.87	93
	white-glass	0.65	0.85	0.74	80
	accuracy			0.85	539
	macro avg	0.85	0.84	0.84	539
	weighted avg	0.86	0.85	0.85	539
4	cardboard	0.90	0.80	0.85	87
	clothes	0.90	0.87	0.88	101
	green-glass	0.98	0.97	0.97	100
	metal	0.71	0.83	0.77	78